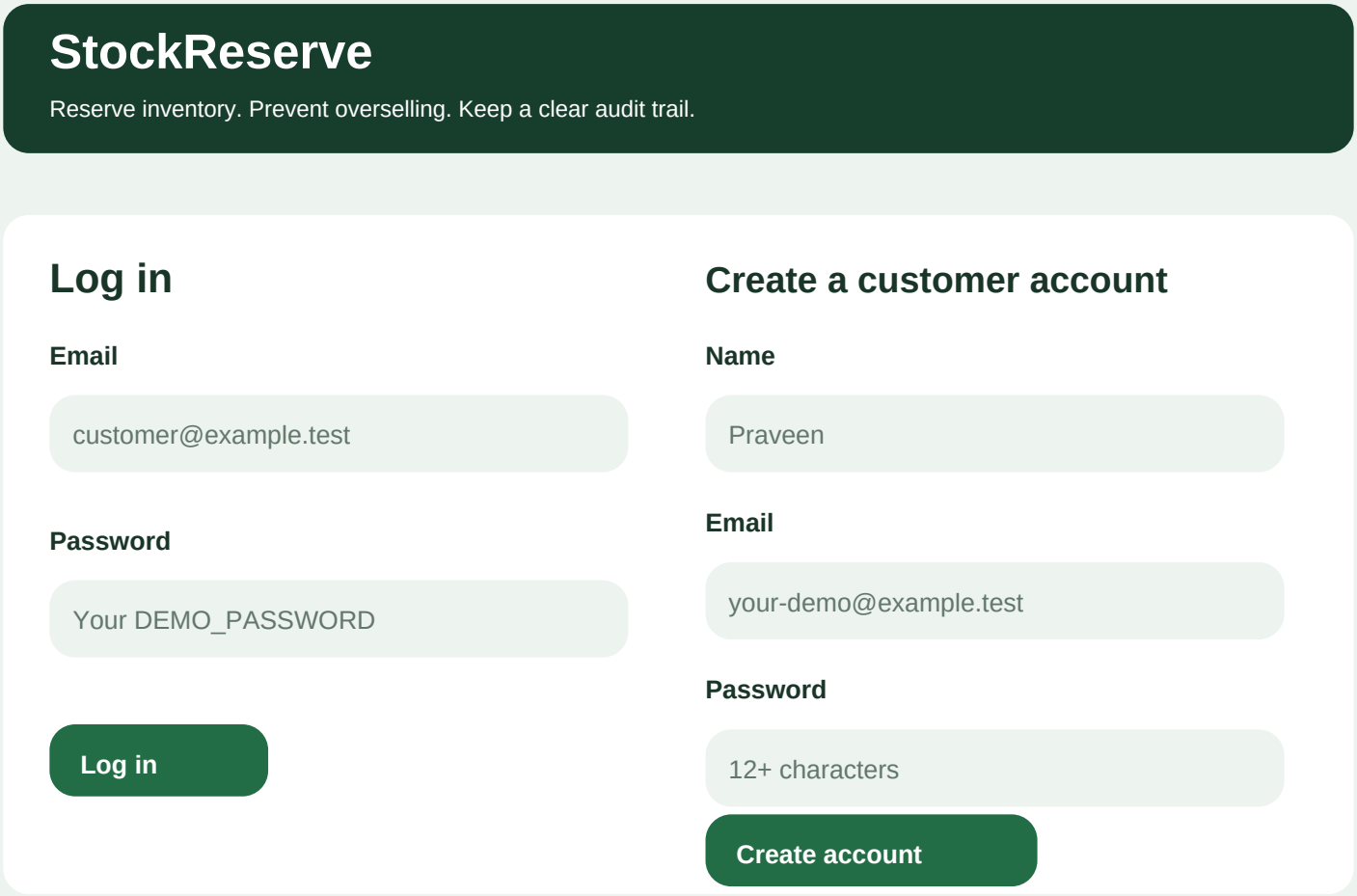
1. Log in or register

Start with the seeded customer account, or create your own customer account.



What happens after login?

The API returns an authentication token. The dashboard uses it to load inventory and your reservations. The token stays in browser memory; reloading the page requires another login.

Customers can reserve products and manage their own reservations. Admins can also create products and view audit events through the API.

Demo login: admin@example.test or customer@example.test. Both use the DEMO_PASSWORD you set during local setup. There is no preconfigured password in the ZIP.

2. Reserve limited stock

Example: reserve 2 Mechanical Keyboards from an initial stock of 12.

StockReserve

Reserve inventory. Prevent overselling. Keep a clear audit trail.

Reserved 2 units. Reference: demo-reservation-001

Available products

Reservations expire after 15 minutes.

Product	Stock	Qty	Action
Mechanical Keyboard	**10**	2	Reserve
Wireless Mouse	**8**	1	Reserve
USB-C Monitor	**3**	1	Reserve

Your reservations

Keyboard / demo reference Qty: 2 **PENDING**

Confirm Cancel

12 available - 2 reserved = 10 available

Choose quantity 2 and click Reserve. The server validates your input, creates a pending reservation, updates stock, and records an audit event in one transaction.

A request for more stock than available returns HTTP 409 and creates no reservation.

3. Confirm, cancel or expire

A reservation can leave the pending state only once.

CONFIRM

Stock stays at 10

The two units remain consumed. Status becomes confirmed. This demo has no checkout or payment processing.

CANCEL

Stock returns to 12

The two held units are released. Status becomes cancelled. Repeating Cancel does not increase stock again.

EXPIRE

Stock returns to 12

After 15 minutes, the expiry command releases the hold. It runs through the scheduler, or when you run it manually.

Duplicate requests are safe

Repeating the same API request with the same Idempotency-Key returns the original reservation: the same reference, with no second stock deduction. Changing the quantity while reusing that key returns a conflict.

The ordinary Reserve button starts a new request after success. To demonstrate replay, repeat the documented curl request using the same key. The UI is not a payment system.

4. Run the real application

PHP 8.2+ and Composer are required. No Node.js build is required.

Local setup

1. Extract the project ZIP and open the stock-reserve-api folder.
2. Run composer install. Copy .env.example to .env.
3. Set DEMO_PASSWORD in .env to at least 12 characters.
4. Run php artisan key:generate.
5. Create database/database.sqlite as an empty file.
6. Run php artisan migrate --seed, then php artisan serve.
7. Open http://localhost:8000 and log in with a seeded account.

Expiry scheduler

Second terminal: php artisan schedule:work

Or run once: php artisan inventory:expire

Verified behavior

16 automated tests passed (48 assertions). Running-server checks passed for login, reservation creation, duplicate replay, stock changes, cancellation, logout, and token revocation.

Tests used SQLite. MySQL concurrency/load testing and an automated browser interaction suite have not been performed. This is a working portfolio demo, not production certification.

Inside the project: README.md, docs/API.md, docs/INTERVIEW.md